

LABORATORIO DE SEGURIDAD OFENSIVA

PRÁCTICA 13

Montaje y Configuración de Apache II

Cifrado SSL, Análisis de Tráfico y Seguridad en Comunicaciones Web

SACZN Cybersecurity

Apache · SSL/TLS · curl · Wireshark · Ubuntu · VirtualBox

1. Introducción

La presente práctica tiene como objetivo configurar el cifrado SSL en el servidor web Apache y analizar mediante dos herramientas — curl y Wireshark — los paquetes que se intercambian entre cliente y servidor, tanto en escenario cifrado (HTTPS) como sin cifrar (HTTP). Se explica además la importancia del tráfico cifrado y las consecuencias de su ausencia.

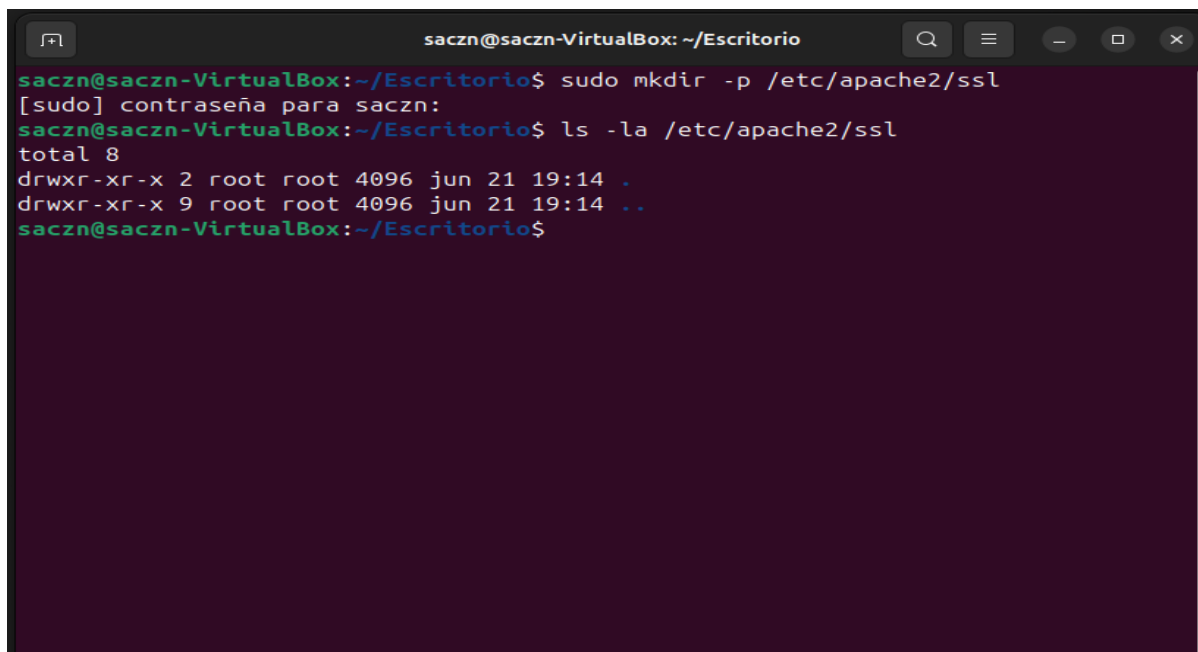
El entorno de trabajo consiste en una maquina virtual Ubuntu 24 en VirtualBox con el servidor Apache configurado para el dominio sacznweb.local, accesible por HTTP (puerto 80) y HTTPS (puerto 443) mediante certificado SSL autofirmado generado con OpenSSL.

2. Configuración y activación del SSL en Apache

2.1 Generación del certificado autofirmado

Se creo el directorio /etc/apache2/ssl y se genero un par clave/certificado RSA de 2048 bits con validez de 365 dias. El campo Common Name se configuro como sacznweb.local, coincidiendo con el ServerName del VirtualHost.

```
$ sudo mkdir -p /etc/apache2/ssl
$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048
-keyout /etc/apache2/ssl/sacznweb.key -out /etc/apache2/ssl/sacznweb.crt
Country: PE | State: LIMA | Org: SACZN Cybersecurity | CN: sacznweb.local
```



```
saczn@saczn-VirtualBox: ~/Escritorio
saczn@saczn-VirtualBox:~/Escritorio$ sudo mkdir -p /etc/apache2/ssl
[sudo] contraseña para saczn:
saczn@saczn-VirtualBox:~/Escritorio$ ls -la /etc/apache2/ssl
total 8
drwxr-xr-x 2 root root 4096 jun 21 19:14 .
drwxr-xr-x 9 root root 4096 jun 21 19:14 ..
saczn@saczn-VirtualBox:~/Escritorio$
```

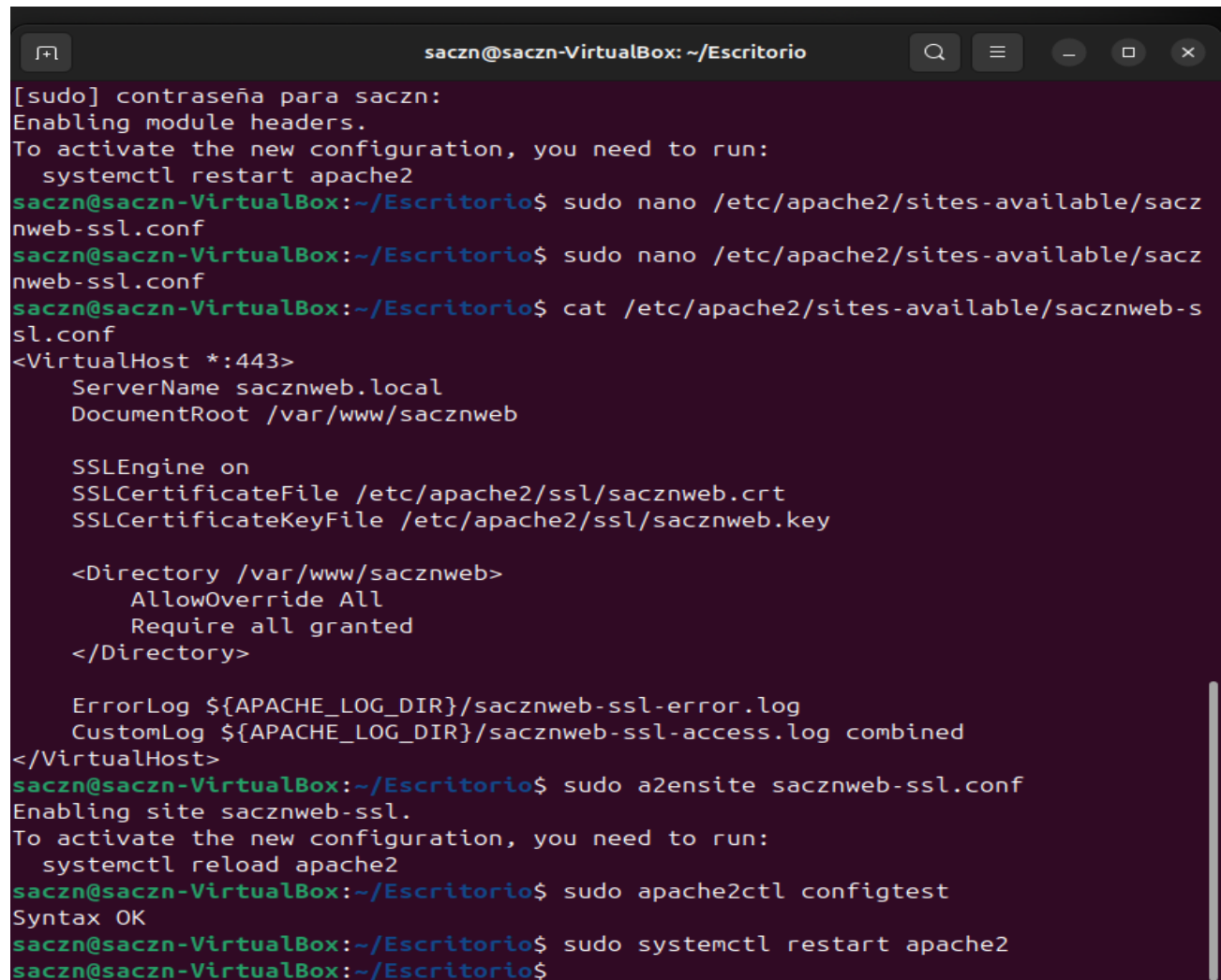
Figura 1. Creación del directorio /etc/apache2/ssl para almacenar el certificado y la clave SSL.

[illegible]

Figura 2. Generación del certificado con OpenSSL: campos del Distinguished Name completados (CN=sacznweb.local).

[illegible]

Figura 3. Verificación con `ls -la: sacznweb.crt (1359 bytes) y sacznweb.key (1704 bytes)` presentes en `/etc/apache2/ssl/`.



```
saczn@saczn-VirtualBox: ~/Escritorio
[sudo] contraseña para saczn:
Enabling module headers.
To activate the new configuration, you need to run:
  systemctl restart apache2
saczn@saczn-VirtualBox:~/Escritorio$ sudo nano /etc/apache2/sites-available/sacznweb-ssl.conf
saczn@saczn-VirtualBox:~/Escritorio$ sudo nano /etc/apache2/sites-available/sacznweb-ssl.conf
saczn@saczn-VirtualBox:~/Escritorio$ cat /etc/apache2/sites-available/sacznweb-ssl.conf
<VirtualHost *:443>
    ServerName sacznweb.local
    DocumentRoot /var/www/sacznweb

    SSLEngine on
    SSLCertificateFile /etc/apache2/ssl/sacznweb.crt
    SSLCertificateKeyFile /etc/apache2/ssl/sacznweb.key

    <Directory /var/www/sacznweb>
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/sacznweb-ssl-error.log
    CustomLog ${APACHE_LOG_DIR}/sacznweb-ssl-access.log combined
</VirtualHost>
saczn@saczn-VirtualBox:~/Escritorio$ sudo a2ensite sacznweb-ssl.conf
Enabling site sacznweb-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
saczn@saczn-VirtualBox:~/Escritorio$ sudo apache2ctl configtest
Syntax OK
saczn@saczn-VirtualBox:~/Escritorio$ sudo systemctl restart apache2
saczn@saczn-VirtualBox:~/Escritorio$
```

Figura 5. Activación del sitio SSL, comprobación de sintaxis (Syntax OK) y reinicio del servicio Apache.

2.3 Verificación en el navegador

Al acceder a <https://sacznweb.local>, Firefox mostro la advertencia MOZILLA_PKIX_ERROR_SELF_SIGNED_CERT, comportamiento esperado al no estar firmado por una CA publica. Tras aceptar el riesgo, el sitio cargo correctamente, confirmando el funcionamiento del cifrado TLS.

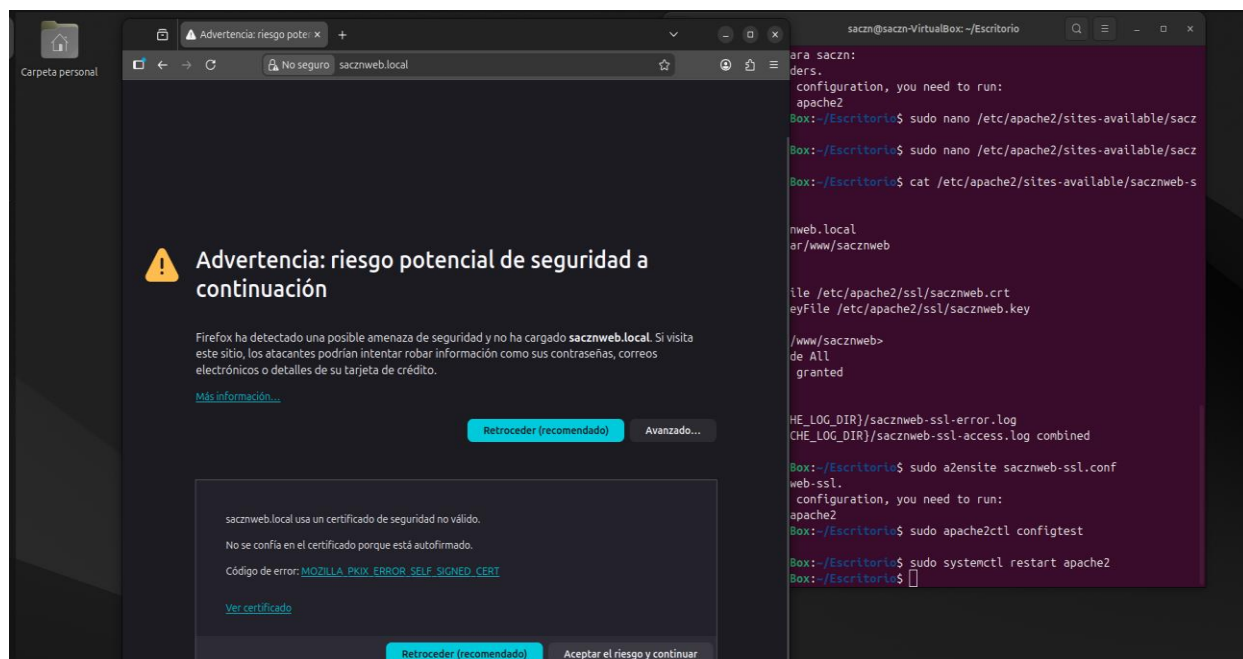


Figura 6. Advertencia de Firefox: `MOZILLA_PKIX_ERROR_SELF_SIGNED_CERT` al acceder a `https://sacznweb.local`.

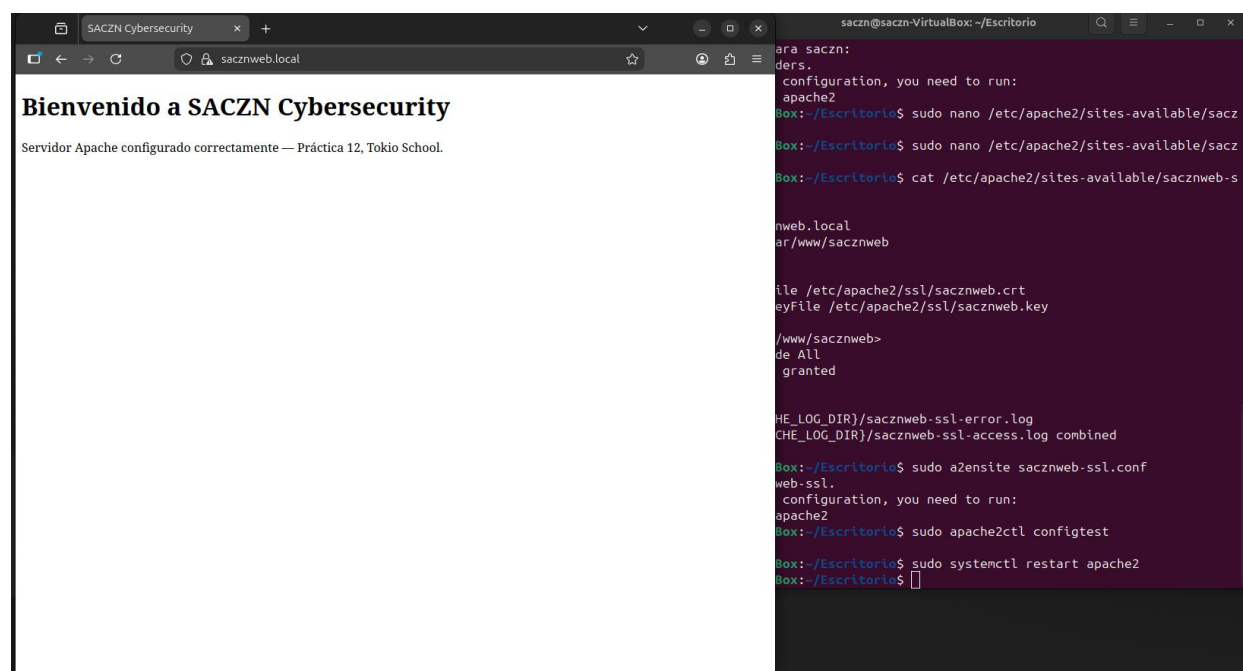


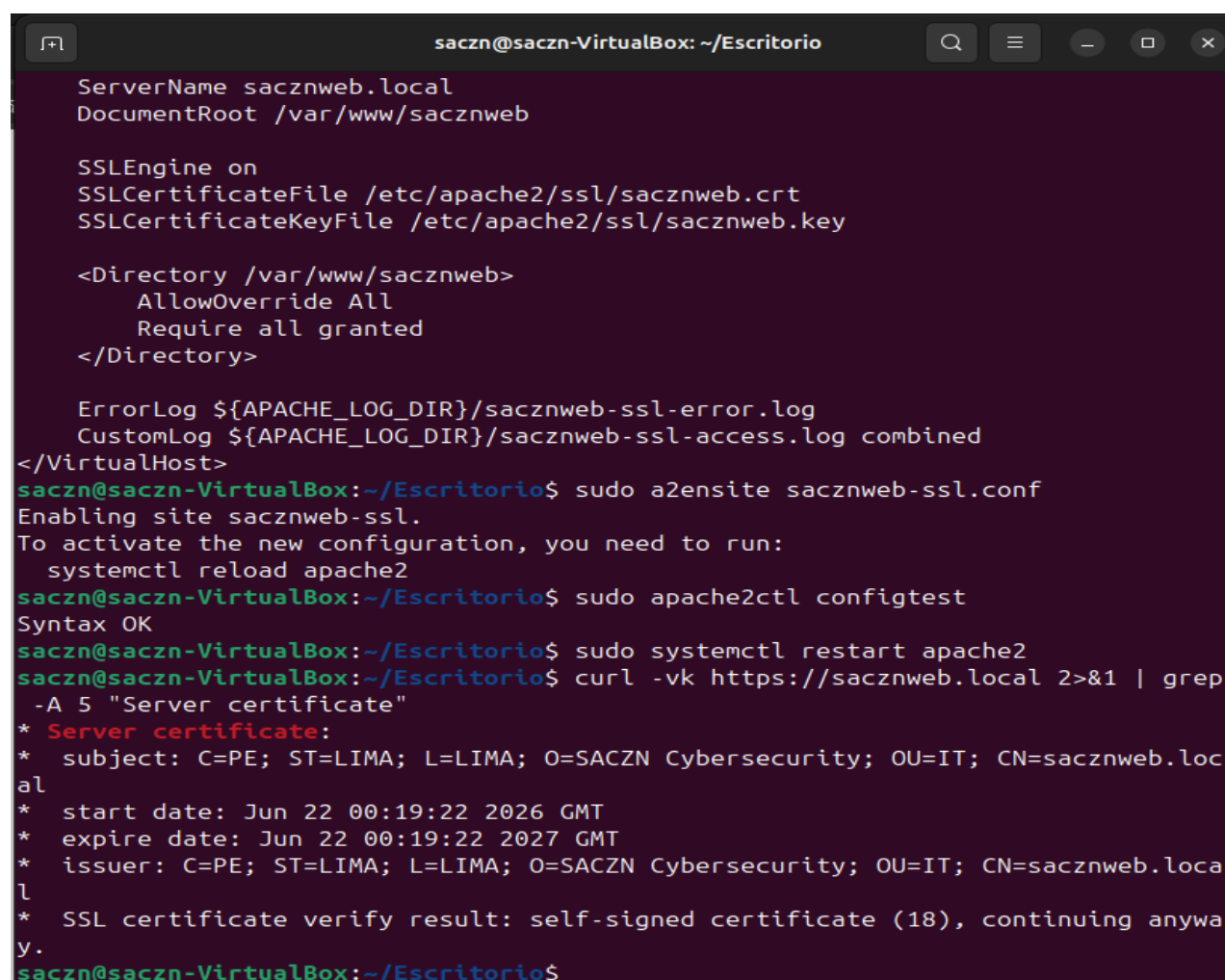
Figura 7. Sitio SACZN Cybersecurity cargando correctamente sobre HTTPS.

3. Análisis del tráfico SSL con curl

La primera herramienta utilizada para analizar el tráfico SSL fue curl, ejecutada desde la misma maquina con los flags `-v` (verbose) y `-k` (ignorar validacion de CA). Esta herramienta permite inspeccionar a nivel de aplicacion los detalles del handshake TLS: version del protocolo, cipher

suite negociada, datos del certificado (subject, issuer, fechas de validez) y el resultado de la verificación.

```
$ curl -vk https://sacznweb.local 2>&1 | grep -A 5 "Server certificate"
* Server certificate:
*  subject: C=PE; ST=LIMA; L=LIMA; O=SACZN Cybersecurity; CN=sacznweb.local
*  start date: Jun 22 00:19:22 2026 GMT
*  expire date: Jun 22 00:19:22 2027 GMT
*  SSL certificate verify result: self-signed certificate (18), continuing
```



```
saczn@saczn-VirtualBox: ~/Escritorio
ServerName sacznweb.local
DocumentRoot /var/www/sacznweb

SSLEngine on
SSLCertificateFile /etc/apache2/ssl/sacznweb.crt
SSLCertificateKeyFile /etc/apache2/ssl/sacznweb.key

<Directory /var/www/sacznweb>
    AllowOverride All
    Require all granted
</Directory>

ErrorLog ${APACHE_LOG_DIR}/sacznweb-ssl-error.log
CustomLog ${APACHE_LOG_DIR}/sacznweb-ssl-access.log combined
</VirtualHost>
saczn@saczn-VirtualBox:~/Escritorio$ sudo a2ensite sacznweb-ssl.conf
Enabling site sacznweb-ssl.
To activate the new configuration, you need to run:
    systemctl reload apache2
saczn@saczn-VirtualBox:~/Escritorio$ sudo apache2ctl configtest
Syntax OK
saczn@saczn-VirtualBox:~/Escritorio$ sudo systemctl restart apache2
saczn@saczn-VirtualBox:~/Escritorio$ curl -vk https://sacznweb.local 2>&1 | grep
-A 5 "Server certificate"
* Server certificate:
*  subject: C=PE; ST=LIMA; L=LIMA; O=SACZN Cybersecurity; OU=IT; CN=sacznweb.local
*  start date: Jun 22 00:19:22 2026 GMT
*  expire date: Jun 22 00:19:22 2027 GMT
*  issuer: C=PE; ST=LIMA; L=LIMA; O=SACZN Cybersecurity; OU=IT; CN=sacznweb.local
*  SSL certificate verify result: self-signed certificate (18), continuing anyway.
saczn@saczn-VirtualBox:~/Escritorio$
```

Figura 8. Verificación del handshake TLS con curl -vk: subject, issuer, fechas de validez y resultado self-signed certificate (18).

La salida de curl confirmó que el certificado autofirmado es reconocido correctamente por el servidor: subject e issuer coinciden (el mismo SACZN Cybersecurity firmó el certificado que emitió), la validez es de exactamente un año (junio 2026 - junio 2027) y el resultado self-signed certificate (18) indica que el cifrado funciona pero que no existe una CA externa que avale la identidad del servidor, lo cual es esperado en entornos de laboratorio.

4. Análisis del tráfico con Wireshark

La segunda herramienta utilizada fue Wireshark 4.2.2, instalado en Ubuntu, que permite capturar y analizar el tráfico de red a nivel de paquete. A diferencia de curl, Wireshark muestra la comunicación completa en tiempo real, incluyendo el handshake TLS y los datos cifrados. Se realizaron dos capturas sobre la interfaz loopback (lo): una para el tráfico HTTPS (puerto 443) y otra para el tráfico HTTP sin cifrar (puerto 80).

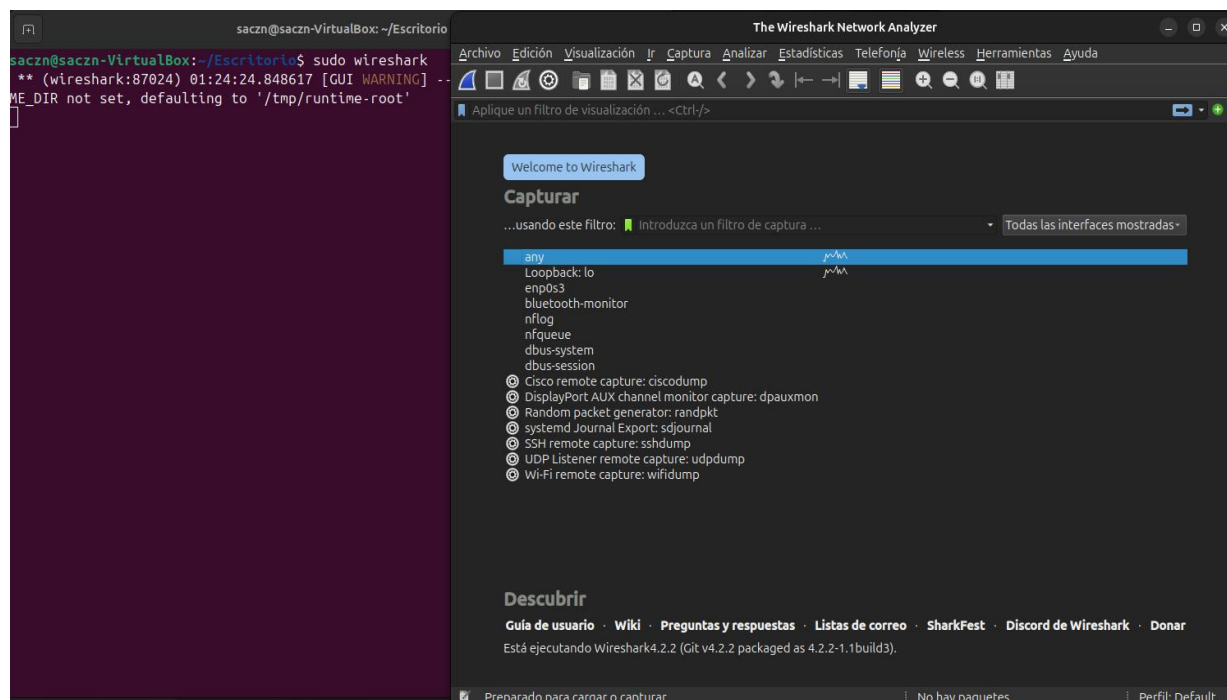


Figura 9. Instalación de Wireshark 4.2.2 en Ubuntu mediante apt.

4.1 Tráfico HTTPS cifrado (puerto 443)

Se aplicó el filtro `tcp.port == 443 and ip.addr == 127.0.0.1` y se recargó `https://sacznweb.local`. La captura registró el handshake TLS completo: Client Hello con SNI=sacznweb.local, Server Hello con Change Cipher Spec, y paquetes de Application Data con el contenido completamente cifrado e ilegible para cualquier observador externo.

```
# Filtro aplicado en Wireshark:  
tcp.port == 443 and ip.addr == 127.0.0.1
```

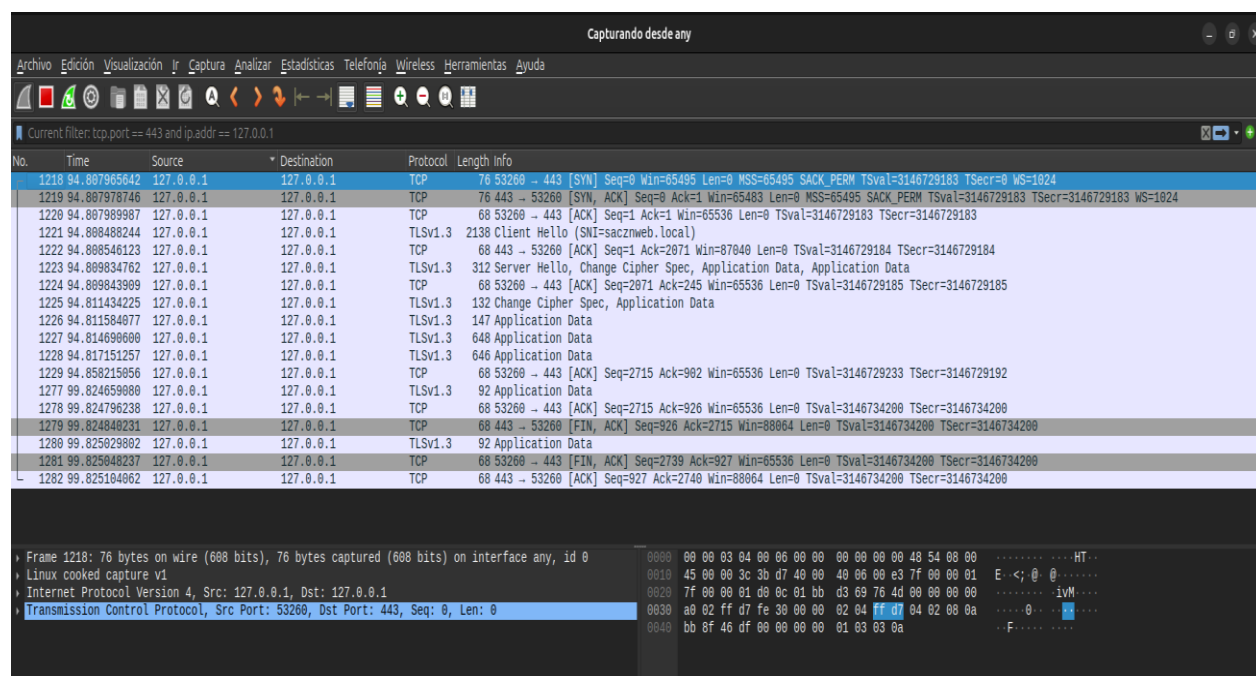



Figura 10. Captura Wireshark (puerto 443): handshake TLS completo. El contenido viaja cifrado como Application Data opaco.

Se analizó en detalle el paquete Client Hello. La expansión del árbol Transport Layer Security mostró la extensión server_name con SNI=sacznweb.local, la version TLS 1.2/1.3 y las 17 cipher suites propuestas, confirmando la correcta negociación a nivel de paquete.

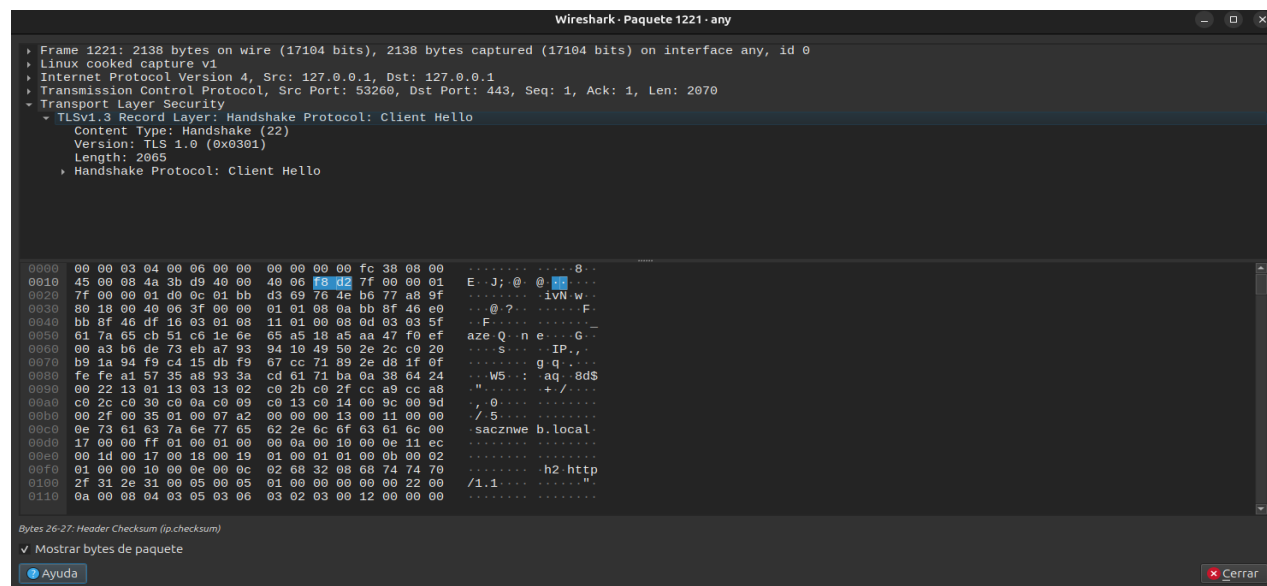


Figura 11. Captura Wireshark del handshake TLS: Client Hello con SNI=sacznweb.local y parametros de negociación.

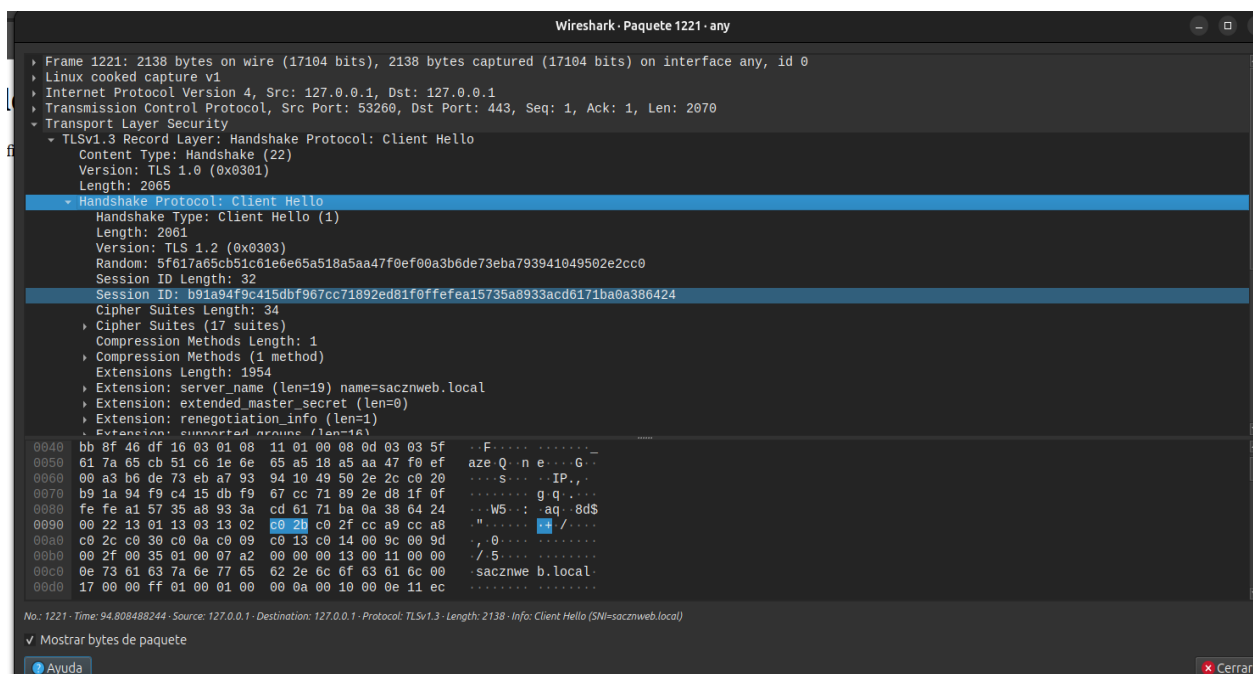


Figura 12. Detalle del paquete Client Hello expandido: extension server_name, TLS 1.2/1.3 y cipher suites negociados.

4.2 Trafico HTTP sin cifrar (puerto 80)

Se aplicó el filtro `tcp.port == 80 and ip.addr == 127.0.0.1` y se accedió a `http://sacznweb.local` (sin HTTPS). La captura mostró los paquetes HTTP completamente en texto plano: el GET / HTTP/1.1 y la respuesta HTTP/1.1 200 OK son perfectamente legibles, sin ninguna capa de cifrado, evidenciando el riesgo de operar sin SSL.

```
# Filtro aplicado en Wireshark:
tcp.port == 80 and ip.addr == 127.0.0.1

# Resultado visible en claro:
GET / HTTP/1.1 -> HTTP/1.1 200 OK (text/html)
```

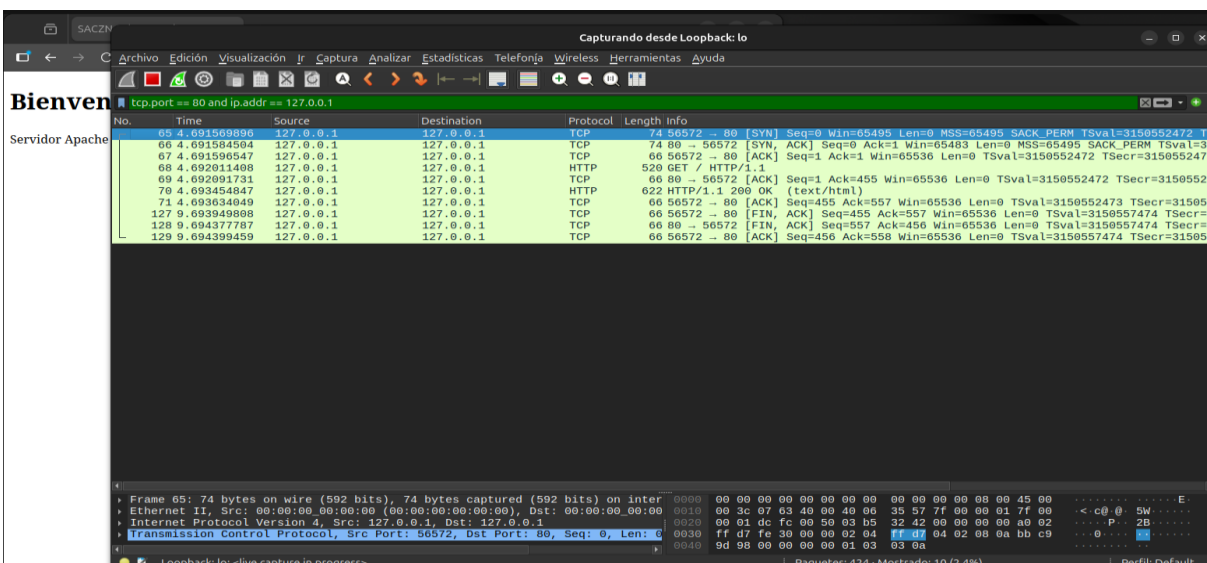


Figura 13. Captura Wireshark (puerto 80): trafico HTTP en texto plano. GET y respuesta 200 OK completamente legibles.

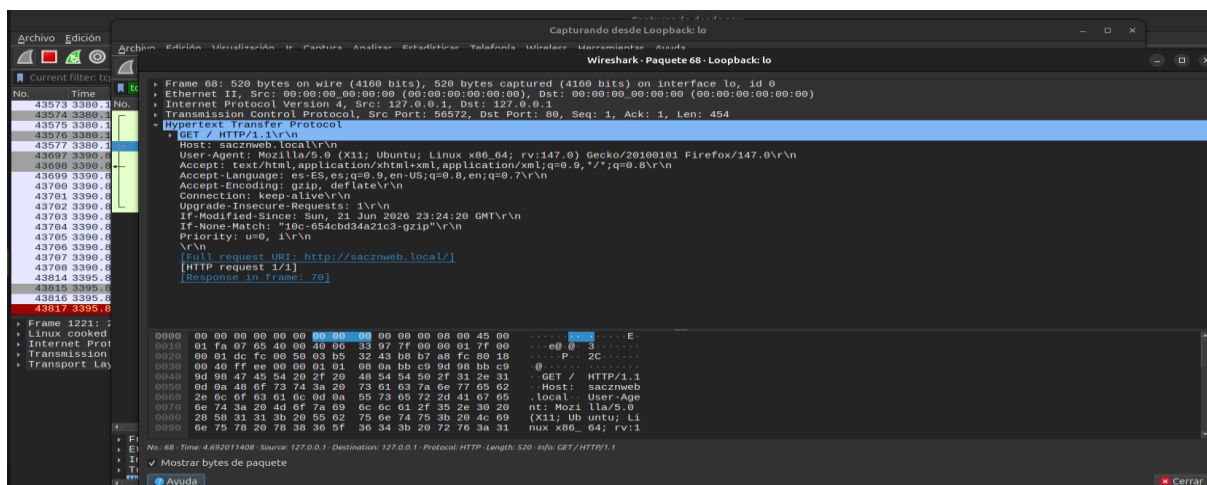


Figura 14. Detalle del paquete GET HTTP/1.1: Host, User-Agent, URI completa y headers visibles sin ningún cifrado.

5. Importancia del cifrado y consecuencias de su ausencia

5.1 ¿Por qué es importante el tráfico cifrado?

El cifrado SSL/TLS garantiza tres pilares fundamentales de la seguridad en las comunicaciones web. La confidencialidad: los datos viajan cifrados y son ilegibles para cualquier tercero que intercepte el tráfico, como se evidencia en las capturas de la sección 4.1 donde el contenido aparece como Application Data opaco. La integridad: el protocolo TLS incluye MACs (Message Authentication Codes) que detectan cualquier modificación de los datos en tránsito. Y la autenticación: los certificados permiten al cliente verificar que se está comunicando con el servidor legítimo.

Adicionalmente, el cifrado protege contra ataques de intermediario (Man-in-the-Middle o MitM), en los que un atacante se interpone entre cliente y servidor para leer o modificar las comunicaciones. En redes locales o publicas, sin TLS este tipo de ataque es trivial con herramientas como Wireshark o Ettercap.

5.2 Consecuencias de la falta de cifrado

Como demuestran las Figuras 13 y 14, sin SSL cualquier observador con acceso a la red puede leer en texto plano todos los datos intercambiados. Las consecuencias concretas incluyen:

Robo de credenciales: formularios de login enviados por HTTP exponen usuario y contraseña en texto plano, capturables trivialmente con Wireshark o tcpdump por cualquier atacante en la misma red.

Session hijacking: las cookies de sesión transmitidas en claro pueden ser capturadas e inyectadas para suplantar la identidad del usuario autenticado sin conocer sus credenciales.

Inyección de contenido: un atacante en la red puede modificar las respuestas HTTP en transito, insertando código malicioso (JavaScript, iframes) en las páginas que recibe el usuario.

Exposición de información del sistema: como muestra la Figura 14, los headers HTTP revelan el sistema operativo, navegador, version, idioma y comportamiento del usuario, datos que un atacante puede usar para preparar ataques dirigidos.

6. Conclusiones

Se ha configurado correctamente el cifrado SSL/TLS en Apache mediante un certificado autofirmado generado con OpenSSL, habilitando HTTPS en **sacznweb.local** a través del puerto 443. El análisis realizado con **curl** confirmó, a nivel de aplicación, los parámetros del certificado (subject, issuer, fechas de validez y condición de certificado autofirmado).

Por su parte, el análisis con **Wireshark** demostró de forma gráfica la diferencia entre el tráfico cifrado (Application Data opaco en TLS) y el tráfico en texto plano (solicitudes GET y encabezados HTTP legibles).

La comparación entre ambos escenarios refuerza la necesidad del uso obligatorio de HTTPS en cualquier servicio web que maneje información de los usuarios. Si bien el certificado autofirmado es adecuado para un entorno de laboratorio, en un entorno de producción debe sustituirse por un certificado emitido por una Autoridad de Certificación (CA) pública, como Let's Encrypt, para evitar advertencias en el navegador y garantizar la confianza de los usuarios.